Module 5

Topic 1 [Method/Function Overloading]

<u>Problem Statement:</u>

Define a class Test where overload a method Sum() to sum numbers sent from main() function.

```
class Test
{
  public:
  //overload Sum() method according to the
     requirement of main()
};
int main(){
  Test t;
  t.Sum(10); //returns 10
  t.Sum(10,20) //return 30
  t.Sum(5.7,20) //return 25.7
  t.Sum(10,2.6) //return 12.6
  t.Sum(10.5,20.7) //return 21.2
}
```

Code:

```
#include<bits/stdc++.h>
using namespace std;
#define newl cout<<endl
class Test
{
   public:
   Test()
   {

   }
   double Sum(double x)
   {
      return x;
   }
   double Sum(double x,double y)
   {
      return x+y;
   }
   double Sum(double x,double y,double z)
   {
      return x+y+z;
   }
};
int main()
{
   Test t;
   cout<<t.Sum(10);newl;
   cout<<t.Sum(10,20);newl;
   cout<<t.Sum(5.7,20);newl;
   cout<<t.Sum(10,2.6);newl;
   cout<<t.Sum(10.5,20.7);newl;
}
```

Topic 2 [**Operator Overloading**]:
<u>Problem Statement:</u>

Suppose in a AC circuit, there are 3 impedances z1=3+j4, z2=4-j3 and z3=j6 are connected in parallel. Now find the current in the circuit if input voltage is 100+j50. Implement **operator overloading** concept for your calculation. Use class **Circuit** and initialize the impedance values (real & img) by a constructor.

```
class Circuit
{
  private:
  int real;
  int img;
  public:
   //write constructor
   //write operator overloaded method
  //write a display method to display real and img
};
int main(){
   Circuit z1(3,4);
  //write required statements to find the current
}
```

Code:

```
#include<bits/stdc++.h>
using namespace std;
#define ll long long int
#define str string
#define newl cout<<endl
class Circuit
{
   private:
   double real;
   double img;
   str s="z0";
   double v1,v2;
   public:
   Circuit()
   {
      s[1]++;
   }
   Circuit(int x,int y)
   {
      real=x;
      img=y;
      s[1]++;
   }
   Circuit operator/(const Circuit &p)
   {
      Circuit result;
      result.real=(100*p.real)-((-p.img)*50);
      result.img=(100*(-p.img))+(50*p.real);
      v1=result.real/(abs(real*real)+abs(img*img));
      v2=result.img/(abs(real*real)+abs(img*img));
      return result;
   }
   void Display()
   {
      cout<<s<<"="<<real;
      if(img>0)
      {
         cout<<"+"<<"j"<<img;newl;
      }
      else
      {
         cout<<"-"<<"j"<<-(img);newl;
      }
```

```
    }
    void current()
    {
        cout<<"I"<<s[1]<<" =";
        cout<<v1;
        if(v2>0)
        {
            cout<<"+"<<"j"<<v2;newl;
        }
        else
        {
            cout<<"-"<<"j"<<-v2;newl;
        }
    }
};
int main()
{
    Circuit z1(3,4),z2(4,-3),z3(0,6);
    z1.operator/(z1);z2.operator/(z2);z3.operator/(z3);
    z1.current();z2.current();z3.current();
}
```

Output:

```
I1 =20-j10
I1 =10+j20
I1 =8.33333-j16.6667

Process returned 0 (0x0)   execution time : 0.040 s
Press any key to continue.
```

Topic 3 [Method/Function Overriding]
Problem statement:

Write a class A with a method **Print()** and a derived class B with method **Print()** overloaded. Now observe the output when following statements are written in the **main()** function

class A
{
public:
void Print()
{
cout<<"Inside Print() of class A"<<endl;
}
};
class B:public A
{
public:
void Print()
{
cout<<"Inside Print() of class B"<<endl;
}
};
	Write Statements inside main()
	i) A a;
	a.Print();
	ii) B b;
	b.Print();
	iii) A a;
	A *p;
	p=&a;
	p->Print();
	iv) B b;
	A *p;
	p=&b;
	p->Print();
	Repeat i)-iv) after writing virtual in front of void Print()

Result:

Main Function	Output

int main()
{
//i)
A a;
a.Print();
}
	Inside Print() of class A

int main()
{
// ii)
B b;
b.Print();
}
	Inside Print() of class B

int main()
{
// iii)
A *p;
p=&a;
p->Print();
	C:\Users\MD... In function 'int main()':
	C:\Users\MD... 32 error: 'a' was not declared in this scope

int main()
{
// iv)
B b;
A *p;
p=&b;
p->Print();
}
	Inside Print() of class A

int main()
{
//After writing virtual::::::
// i)
A a;
a.Print();
}
	Inside Print() of class A

int main()
{
//After writing virtual::::::
// ii)
B b;
b.Print();
}
	Inside Print() of class B

int main()
{
//After writing virtual::::::
// iii)
A *p;
p=&a;
p->Print();
}
	C:\Users\MD... In function 'int main()':
	C:\Users\MD... 50 error: 'a' was not declared in this scope

int main()
{
//After writing virtual::::::
// iv)
B b;
A *p;
p=&b;
p->Print();
}
	Inside Print() of class B

Topic 4 [Pure Virtual Function]

<u>Problem statement:</u>

Modify the class defined in Topic 3 executes the following statements i)-iv) and observe the output:

class A{
public:
virtual void Print()=0;
}
};
class B:public A{
public:
void Print(){
cout<<"Inside Print() of class B"<<endl;
}
};
	Write Statements inside main()
	i) A a;
	a.Print();
	ii) B b;
	b.Print();
	iii) A a;
	A *p;
	p=&a;
	p->Print();
	iv) B b;
	A *p;
	p=&b;
	p->Print();

Result:

Main Function	Output

int main()
{
// i)
A a;
a.Print();
}
	C:\Users\MD... In function 'int main()':
	C:\Users\MD... 52 error: cannot declare variable 'a' to be of abstract type 'A'

int main()
{
// ii)
B b;
b.Print();
}
	Inside Print() of class B

int main()
{
// iii)
A *p;
p=&a;
p->Print();
}
	C:\Users\MD... In function 'int main()':
	C:\Users\MD... 59 error: 'a' was not declared in this scope

int main()
{
// iv)
B b;
A *p;
p=&b;
p->Print();
}
	Inside Print() of class B

Topic 5 [Friend Function]
<u>Problem Statement:</u>

Using the following class, write three friend functions

i) Add(): Assign value to the data member x
ii) IncX() : Increase the value of x by m
iii) DecX() : Decrease the value of x by n

```
class A
{
  private:
  int x;
  public:
    //Prototype of friend functions
};
   //write body of friend functions
   int main()
{
  //call these methods
}
```

Code:

```
#include<bits/stdc++.h>
using namespace std;
#define ll long long int
#define newl cout<<endl
class A
{
   private:
   int x;
   public:
   A()
   {

   }
   friend void add(A &p,int a);
   friend void Inc(A &p, int m);
   friend void Dec(A &p,int n);
   void Display()
   {
      cout<<x;newl;
   }
};
void add(A &p,int a)
{
   p.x=a;
}
void Inc(A &p,int m)
{
   p.x+=m;
}
void Dec(A &p,int n)
{
   p.x-=n;
}
int main()
{
   A a;
   add(a,10);cout<<"After assigning the value---> ";
   a.Display();
   Inc(a,15);cout<<"After increasing the value-->";
   a.Display();
   Dec(a,5);cout<<"After decreasing the value-->";
   a.Display();
}
```

Output:

```
After assigning the value--->10
After increasing the value-->25
After decreasing the value-->20

Press any key to continue . . .
```